

Can LLMs Normalize Databases?

A Benchmark and Multi-Agent Framework for Schema Normalization

Dong-Jae Koh* Huisu Kim* SeongHwan Yoon*

Lasse M. Jantsch Chun-Hee Lee Seonghyeon Lee Young-Kyoon Suh†

School of Computer Science and Engineering, Kyungpook National University, South Korea
{djkoh, kimisu0712, shyoon0214, lassejantsch, chunhee, sh0416, yksuh}@knu.ac.kr

Abstract

Large Language Models (LLMs) are increasingly used to generate structured outputs, but their reliability remains unclear when those outputs must satisfy database-level constraints. We study this issue through database normalization, involving reasoning about functional dependencies, lossless join decompositions, and inter-table constraints. We introduce a *Database Normalization Benchmark* (DNBENCH), comprising 3,275 samples for evaluating LLM-driven database normalization from 1NF to BCNF. DNBENCH uses a three-axis protocol to measure *semantic equivalence*, *structural accuracy*, and *logical validity*. Across *Single*, *Complex*, and *Real World* levels, DNBENCH uncovers recurring failures in dependency inference, schema decomposition, and inter-table constraint reconstruction. We further propose *Multi-Agent Reasoning for Schemas* (MARS), which separates evidence extraction, violation diagnosis, and decomposition planning from schema generation and verification. MARS improves the DNB-SCORE by 82.0% over the single-prompt baseline. All artifacts will be released upon acceptance.

1 Introduction

Large language models (LLMs) (Brown et al., 2020) have been explored for structured-data reasoning and database-related tasks. These tasks require outputs that satisfy formal constraints. In this context, LLMs’ ability to follow natural-language instructions, reason over tabular data, and generate structured outputs makes them a promising approach for automating database workflows.

Normalizing databases using LLMs highlights this challenge because it requires both reasoning over dependencies and generating schemas that satisfy formal relational constraints. Database normalization is a foundational technique in relational

data management (Banks et al., 1970), reducing redundancy, preventing update anomalies, and enhancing schema management by organizing data according to functional dependencies (FD). When a database schema is not properly normalized, it introduces data inconsistency, unnecessary query costs, and degraded data quality (Kent, 2000). Moreover, normalization is difficult to automate because it requires reasoning over functional dependencies, candidate keys, lossless-join decompositions, and dependency preservation (Codd, 1971).

Despite this importance, existing studies provide limited evidence for whether LLMs can handle database normalization. NormTab (Nahid and Rafiei, 2024) and TABARD (Choudhury et al., 2025) address normalization-related issues in table-cleaning or anomaly-detection settings, rather than schema normalization itself. Miffie (Jo et al., 2025) targets 1NF-3NF normalization with an LLM generator-verifier loop, but its verification lacks explicit schema-level checks for properties such as lossless join, key validity, dependency reasoning, and foreign-key connectivity. Consequently, the reliability of LLM-based database normalization remains insufficiently understood.

To address these concerns, we introduce a comprehensive *Database Normalization Benchmark* (DNBENCH) for evaluating end-to-end LLM-driven database normalization. DNBENCH provides 3,275 denormalization samples from realistic relational schemas, covering 1NF-to-BCNF violations with gold labels and decomposition targets. DNBENCH further proposes a three-axis evaluation protocol that measures *semantic equivalence*, *structural accuracy*, and *logical validity*, combining them into a unified *DNB-Score* to be presented in Section 3. We further design *Multi-Agent Reasoning for Schemas* (MARS), a framework that decomposes the normalization process into several subtasks and distributes them across multiple LLM agents that collaborate to perform normalization.

* Equal Contributions

† Corresponding author

Method	LLM Based	Normalization					Evaluation		Benchmark Provided
		1NF	2NF	3NF	BCNF	Complex	Schema	Expl.	
RDBNorma (2011)	✗	✗	✓	✓	✗	✗	✗	✗	✗
EDNA (2013)	✗	✗	✓	✓	✓	✗	✗	✗	✗
NormTab (2024)	✓	✓	✗	✗	✗	✗	✗	✗	✗
TABARD (2025)	✓	✗	✗	✗	✗	✗	✓	✗	✓
Miffie (Dual-LLM SR) (2025)	✓	✓	✓	✓	✗	✗	✓	✗	✗
DNBENCH (ours)	✓	✓	✓	✓	✓	✓	✓	✓	✓

Table 1: Comparison of DNBENCH with related work methods. **Normalization** groups 1NF to BCNF support and Complex (multi-violation) handling. **Evaluation** groups schema and explanation level scoring.

Our experiments characterize the current state of automatic database normalization using LLMs. Our DNBENCH enables quantification of the quality of generated DDLs on real-world databases. Also, we empirically demonstrate that our MARS approach produces, on average, 82.0% more reliable DDLs while preserving inter-table constraints compared to naive prompting. Our contributions are as follows:

- We introduce DNBENCH, a 3,275-sample benchmark for evaluating LLM-based database normalization, covering 1NF-to-BCNF violations.
- We propose a three-axis evaluation protocol that assesses semantic equivalence, structural accuracy, and logical validity of generated schemas.
- We benchmark modern LLMs on DNBENCH and identify major failure patterns, including unreliable FD inference, invalid decomposition, and weak inter-table constraint reconstruction.
- We propose MARS, a multi-agent normalization framework that improves DNB-SCORE by 82.0% over the baseline.

2 Related Work

Table 1 positions DNBENCH relative to prior normalization tools and LLM-based methods. Existing work falls into two broad categories: *traditional tools* that assist users in applying normalization rules and *LLM-based systems* that address related table-cleaning or limited normalization tasks. However, neither category provides a comprehensive benchmark for evaluating whether LLMs can reliably perform Database normalization. DNBENCH fills this gap with a comprehensive normalization benchmark and a multi-faceted evaluation protocol.

2.1 Traditional Normalization Tools

Traditional normalization tools are primarily rule-based or semi-automatic systems designed for user-assistance purposes. While useful for automating normalization procedures, they are not designed as benchmarks; they typically cover only limited settings and lack quantitative metrics to measure normalization quality.

Dongare et al. (2011) introduces RDBNorma, a semi-automated tool for normalizing schemas up to 3NF. RDBNorma represents schemas and functional dependencies using linked lists and checks generated outputs against expected decompositions. However, its evaluation focuses mainly on runtime and memory efficiency rather than on semantic preservation, lossless decomposition, key validity, or violation diagnosis.

Levandowski et al. (2013) presents EDNA, a semi-automated tool that supports normalization up to BCNF given user-specified functional dependencies. EDNA generates rule-based normalized schemas from FDs, but it does not provide a quantitative protocol for evaluating normalization or robustness across diverse schemas and complex violations.

2.2 LLM-based Normalization

Recent work has begun to explore the use of LLMs for table and schema normalization. However, existing approaches either target different objectives or evaluate narrower forms of normalization.

Nahid and Rafiei (2024) propose NormTab, an LLM-based framework for normalizing human-readable web tables, such as Wikipedia tables. NormTab performs value-level and structural cleaning, but its goal is to improve downstream SQL-based question answering and fact verification. It is therefore closer to table preprocessing rather than database normalization: it does not evaluate whether LLMs can reason

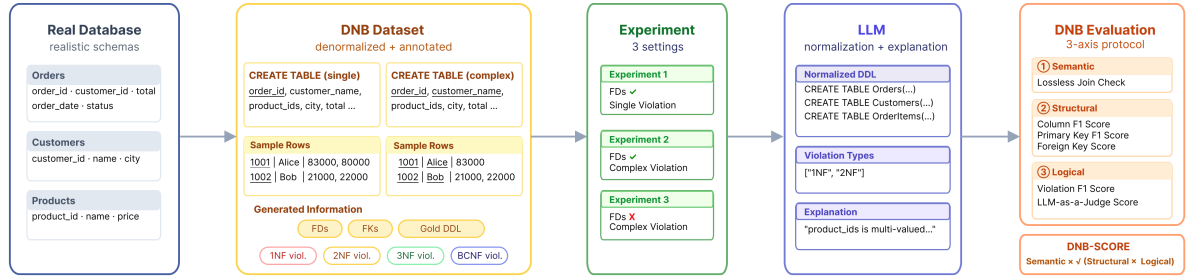


Figure 1: Overview of the database normalization benchmark (DNBENCH) pipeline. DNBENCH generates controlled denormalized schemas with gold violation labels and decompositions, then evaluates LLM-generated DDL, violation types, and explanations along semantic, structural, and logical axes.

over functional dependencies, preserve lossless decompositions, or construct valid normalized schemas.

TABARD (Choudhury et al., 2025) evaluates LLMs on anomaly detection in tabular data, where models identify anomalous cells under different prompting strategies. Although relevant to tabular reasoning, TABARD does not target normal-form violations or schema decomposition. As a result, TABARD does not address whether an LLM can diagnose dependency violations or produce a structurally valid decomposition.

Most relevant to our work, Miffie (Jo et al., 2025) proposes a Dual-LLM self-refinement framework for 1NF-to-3NF normalization. Miffie uses one LLM to generate decompositions and another to provide feedback on remaining violations. However, its verifier is also LLM-based and does not include explicit schema-level checks for lossless join, key validity, or foreign-key connectivity. In contrast, DNBENCH combines LLM-based semantic judgment with explicit structural and logical checks to measure whether decompositions are semantically equivalent, structurally accurate, and logically valid.

3 Database Normalization Benchmark

Figure 1 shows the proposed database normalization benchmark (DNBENCH). DNBENCH transforms real-world relational databases into controlled denormalized samples annotated with gold violation labels and gold decomposition targets. Given each sample, an LLM generates normalized data definition language (DDL), violation labels, and explanations. DNBENCH evaluates these outputs along semantic, structural, and logical axes.

DNBENCH consists of two components: (i) a denormalization pipeline that constructs controlled normal-form violations from real-world relational

Dataset Attribute	Spider	BIRD
<i>Original datasets</i>		
Number of DBs	200	95
Number of domains	138	37
Number of tables	1,020	694
<i>DNBENCH dataset</i>		
Dev DBs	116	40
Test DBs	74	24
Dev samples	1,495	495
Test samples	900	385
Total samples	2,395	880

Table 2: Descriptive statistics for the source Spider and BIRD datasets and the resulting DNBENCH dataset.

schemas, and (ii) a three-axis evaluation protocol that measures semantic equivalence, structural accuracy, and logical validity. These three scores are combined into a unified DNB-SCORE to summarize overall normalization quality.

3.1 Dataset Construction

We construct DNBENCH using two Text-to-SQL corpora. Spider (Yu et al., 2018) and BIRD (Li et al., 2023) provide realistic relational schemas with primary-key (PK) and foreign-key (FK) annotations. These schemas serve as sources for controlled denormalization: DNBENCH injects and labels normal-form violations into existing relational designs while preserving key constraints. After construction, three database experts audited the generated samples and confirmed that the intended 1NF-to-BCNF violation patterns, chain-rule labels, and gold decompositions were correctly represented. Table 2 summarizes the source DBs and the resulting DNBENCH dataset. Our dataset preserves realistic schema structure while introducing controlled normal-form violations and gold decomposition targets.

We construct the dataset through a three-stage denormalization pipeline: (1) FK-aware subsampling,

(2) hybrid discovery–synthetic violation injection, and (3) chain-rule labeling.

Stage 1: Foreign Key-aware Subsampling. We first downsample each source database while preserving FK constraints. The sampling procedure traverses the FK graph from leaf tables upward, samples up to n tuples from each source table, and expands the selected key sets through FK and self-FK closures. It then applies a global size cap and removes remaining FK violations through cascade cleanup. This produces compact database instances that remain referentially valid. Full details and the complete algorithm are provided in Appendix A.3.

Stage 2: Discovery and Synthesis of Violations. For each source schema, we identify normal-form violation patterns: multivalued cells for 1NF, partial dependencies for 2NF, transitive dependencies for 3NF, and non-superkey determinants for BCNF.

Because real schemas often contain entangled rather than isolated violations, discovery alone is insufficient for controlled evaluation. We therefore complement it with deterministic synthesizers for 2NF, 3NF, and BCNF. These synthesizers incur violations through PK splitting, transitive-dependency injection, and candidate-key augmentation, respectively. We compose synthesized violations in reverse chain-rule order—BCNF, 3NF, 2NF, and 1NF—so that lower-form injections do not erase previously introduced higher-form violations. This construction yields composite samples in which all intended violations remain observable.

Stage 3: Chain-rule Labeling. Each sample includes two gold decomposition targets: a *single* target resolving only the earliest violation and a *combined* target resolving the full violation chain.

The composite synthesizers in Stage 2 ensure that fixing a lower-form violation does not automatically remove higher-form violations. Thus, each violation must be handled explicitly, making the *combined* target distinct from the *single* target. This enables DNBENCH to test whether a model fixes only the earliest violation or normalizes the full violation chain. Each sample also includes an expected FK list, allowing FK correctness to be scored without re-parsing generated DDL.

3.2 Three-Axis Evaluation Protocol

We evaluate each LLM prediction along three complementary axes: *semantic equivalence*, *structural accuracy*, and *logical validity*. These axes

capture whether a generated decomposition preserves the original data, matches the expected schema structure, and provides correct normalization reasoning. We aggregate the three axes into a unified DNB-SCORE. The full prompts for both the LLM generator and the LLM-as-a-Judge are provided in Appendix H.

Criterion 1. Semantic Evaluation The semantic axis tests whether the generated decomposition satisfies the lossless-join property with respect to the original table. Specifically, the natural join of the generated relations must reconstruct the input table without introducing spurious tuples. We report this check as a binary score. Since a lossless join is a necessary condition for valid normalization, the semantic score acts as a gate in DNB-SCORE: any semantic failure reduces the final score to zero.

Criterion 2. Structural Evaluation The structural axis measures how closely the generated DDL matches the gold decomposition. We parse the LLM-generated DDL into relational schema objects and compute three metrics:

- **Column F1:** F1 between the generated and gold column sets.
- **Primary Key F1:** F1 between the generated and gold primary-key sets.
- **Foreign Key Score:** The product of FK matching and FK validity:
 - **FK F1:** F1 between the generated and gold foreign-key sets.
 - **FK Connected Score:** A binary indicator of whether generated FK constraints validly reference extant tables and columns.

The final structural score is the mean of Column F1, Primary Key F1, and Foreign Key Score.

Criterion 3. Logical Evaluation The logical axis evaluates whether the model correctly diagnoses the normalization problem and justifies its decomposition. It combines two components: (i) an LLM-as-a-Judge score for the generated schema and explanation, and (ii) Violation F1 for the predicted normal-form violation labels.

The judge LLM rates each prediction along three dimensions:

- **Logical Coherence:** whether the generated decomposition is consistent with the intended normalization logic and gold DDL.
- **Explanation of Schema Alignment:** whether the explanation is internally consistent with the

generated DDL and declared violation labels.

- **Explanation Quality:** whether the explanation uses normalization concepts correctly and presents the reasoning clearly and concisely.

We use openai/gpt-oss-20b (Agarwal et al., 2025) as the judge model. We select this model because the Judge’s Verdict benchmark identifies it as a human-like judge, showing strong agreement with human annotators without exceeding the natural range of human judgment variation (Han et al., 2025). This choice is suitable for evaluating normalization explanations, where excessive consistency may overlook valid judgment nuances. We further validate the judge on our task through an expert-agreement study; details are provided in the Appendix E.

Violation F1 compares the violation labels predicted by the model against the gold violation labels. For each input table, we compute F1 over violation types, measuring whether the model correctly identifies which violations are present.

The final score is the mean of the average judge score across three dimensions and Violation F1.

DNB-SCORE. We aggregate the three axes as:

$$\text{DNB-SCORE} = \text{Semantic} \times \sqrt{\text{Structural} \times \text{Logical}}.$$

Inspired by BLEU (Papineni et al., 2002), DNB-SCORE uses the geometric mean of the structural and logical scores when the semantic score is 1. This aggregation rewards schemas that satisfy both complementary criteria, preserves the original score scale, and penalizes imbalance more strongly than an arithmetic mean.

4 Database Normalization Analysis

We evaluate LLMs’ database normalization capability using our benchmark dataset and evaluation protocol. Our experiments are designed to measure two core capabilities required for database normalization: (i) identifying and reasoning over functional dependencies (FDs), and (ii) resolving multiple co-occurring violations in the same schema.

4.1 Experimental Setup

We use the DNBENCH test split generated by the pipeline in Section 3.1, as summarized in Table 2. Each sample contains a denormalized input table, row samples, gold violation labels, gold decompositions, and expected schema constraints. We test four representative LLMs spanning dense

Experiment	Target Scope	FDs
Exp. 1: Single	Earliest violation only	Provided
Exp. 2: Complex	All violations	Provided
Exp. 3: Real World	All violations	Not provided

Table 3: Distinctions among the three experiments.

and sparse mixture-of-experts (MoE) architectures across different scales. Each setting is evaluated under both zero-shot and few-shot prompting, yielding six configurations. More details on models and prompts are provided in Appendices B and H.

We design three experimental settings for a comprehensive evaluation under different normalization scopes and FD availability (Table 3).

Experiment 1: Single Reasoning

Experiment 1 tests whether a model can resolve the earliest violated normal form given explicit FD evidence. Although an input may contain a chain of violations, the model is asked to fix only the first violation: for example, 1NF in a 1NF–2NF–3NF–BCNF chain, or 2NF in a 2NF–3NF–BCNF chain. This setting isolates the model’s ability to apply a single normalization rule.

Experiment 2: Complex Reasoning

Experiment 2 extends the task to the full violation chain while still providing explicit FD evidence. The model must identify all relevant violations and produce a valid multi-step decomposition. This setting evaluates whether the model can handle interactions among multiple normalization rules.

Experiment 3: Real World Reasoning

Experiment 3 removes explicit FDs and requires the model to infer them from context. The input includes a table, row samples, and natural-language business rules, but it withholds formal FD annotations. The model must infer latent FDs, identify the full set of violations, and generate a normalized schema. This setting reflects practical normalization scenarios in which dependency information is not directly provided.

4.2 Baseline Result and Analysis

DNBENCH identifies two valuable findings when using an LLM to normalize relational databases.

Findings 1. FD extraction is the main bottleneck in LLM-based database normalization. Our results validate that all four models degrade most sharply in the *Real World* setting, where FDs are not explicitly provided (Table 4). Averaged across

Model	Single			Complex			Real World		
	Zero-shot	Few-shot	Avg	Zero-shot	Few-shot	Avg	Zero-shot	Few-shot	Avg
Llama 3.3 70B	0.380 [†]	0.451	0.416	0.371	0.327 [†]	0.349 [†]	0.196 [†]	0.212 [†]	0.204 [†]
Gemma3 27B	0.353	0.341 [†]	0.347	0.361 [†]	0.366	0.363	0.158	0.220	0.189
Qwen3-30B	0.489	0.304	0.397 [†]	0.340	0.209	0.274	0.253	0.209	0.231
Mixtral 8x7B Instruct	0.146	0.159	0.153	0.155	0.150	0.152	0.132	0.120	0.126
Overall	0.342	0.314	0.328	0.307	0.263	0.285	0.185	0.190	0.188

Table 4: Average DNB-SCORE ([†]) on the DNBENCH test split by model and evaluation settings; bold and [†] mark the best and second-best scores in each column.

Model	Semantic	Structural			Logical	
	lossless join	Column F1	PK F1	FK Score	Violation F1	LLM Judge
Llama 3.3 70B	0.574 [†]	0.909 [†]	0.637	0.174	0.639 [†]	0.383
Gemma3 27B	0.634	0.921	0.605	0.140	0.502	0.334
Qwen3-30B	0.568	0.854	0.611 [†]	0.158 [†]	0.678	0.361 [†]
Mixtral 8x7B Instruct	0.412	0.742	0.492	0.104	0.437	0.227
Overall	0.547	0.857	0.586	0.144	0.564	0.326

Table 5: Per-model DNB-SCORE component breakdown, averaged over the DNBENCH test split and all six configurations. **Bold** indicates the best score per column, and [†] the second-best.

models, DNB-SCORE drops modestly from 0.328 in *Single* to 0.285 in *Complex*, but declines more sharply to 0.188 in *Real World*. This pattern indicates that, once FDs are given, models can apply normalization rules to some extent. The harder problem is inferring reliable FDs from data samples and natural-language business rules.

Findings 2. LLMs perform better at violation detection than at schema reconstruction. Table 5 separates violation diagnosis from schema generation. Violation F1 reaches up to 0.678, indicating that models can often identify which normal forms are violated. However, structural performance remains much lower, especially for inter-table constraints: FK Score ranges only from 0.10 to 0.17 across the four models. Thus, the central challenge is not merely recognizing that a decomposition is needed but producing a normalized schema with a valid PK and FK structure. In other words, LLMs are stronger at local violation diagnosis than at global schema reconstruction.

Furthermore, our in-depth per-violation analysis shows that most models commonly struggle with multiple violations, BCNF reasoning, and already-normalized inputs, even though performance varies across violation paths (Table 7 in the Appendix). The NONE cases are particularly revealing, as models sometimes introduce unnecessary decompositions even when no normalization is required.

Overall, current LLMs face three main chal-

lenges in database normalization: (i) unreliable FD inference in realistic settings, (ii) weak reconstruction of valid inter-table constraints, and (iii) a tendency to over-normalize already valid schemas. These findings motivate a structured normalization framework that separates reasoning stages and incorporates explicit verification.

5 Multi-Agent Reasoning for Schemas

The baseline analysis shows that LLM-based normalization often fails because several distinct reasoning steps are compressed into a single prompt. A model must infer FDs, diagnose normal-form violations, plan decompositions, and generate SQL DDL in one pass. Errors from earlier steps can propagate directly to the final schema without intermediate correction.

Multi-agent LLM frameworks address this issue by decomposing complex tasks across role-specialized agents that exchange and verify intermediate outputs. AutoGen (Wu et al., 2023) provides general infrastructure for orchestrating conversational agents, while MetaGPT (Hong et al., 2023) and ChatDev (Qian et al., 2024) demonstrate the effectiveness of role-based agent pipelines. Motivated by these findings, we adapt role specialization and verifier feedback to database normalization, where each step must satisfy dependency, foreign-key, and lossless-join requirements.

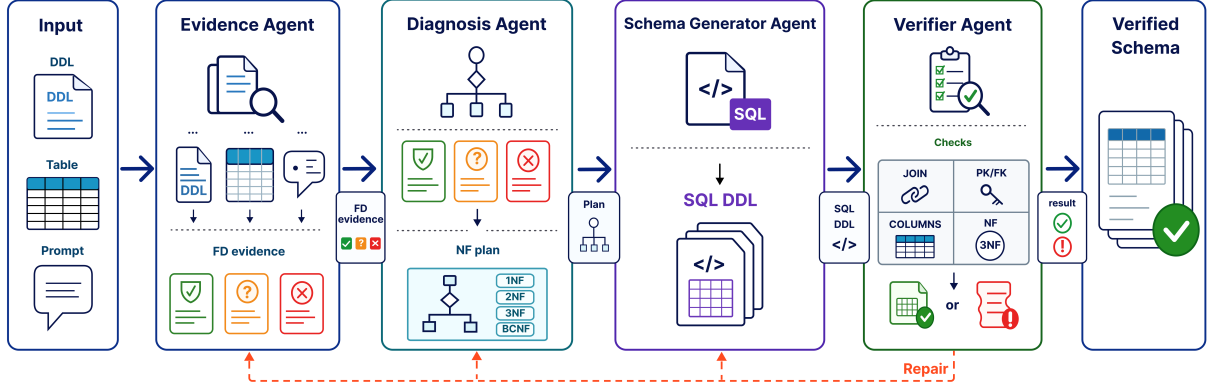


Figure 2: Overview of MARS framework—Evidence, Diagnosis, Schema Generator, and Verifier agents—together with the repair loop triggered by verification failures.

5.1 MARS Framework

We propose Multi-Agent Reasoning for Schemas (MARS), which decomposes LLM-based database normalization into four stages: (i) *evidence extraction*, (ii) *violation diagnosis and decomposition planning*, (iii) *schema generation*, and (iv) *deterministic verification* (Figure 2).

Evidence Agent. The Evidence Agent extracts functional-dependency (FD) evidence from the input schema, row samples, and optional FD annotations. It treats provided FDs as direct evidence or infers candidate FDs from data, then classifies them as reliable, hypothetical, or rejected for downstream agents.

Diagnosis Agent. The Diagnosis Agent identifies violated normal forms and the FDs responsible for those violations, prioritizing reliable FDs over hypothetical ones. It then builds a stepwise decomposition plan that specifies the key dependencies, decomposition order, and target relations.

Schema Generator Agent. The Schema Generator Agent emits SQL DDL for the planned decomposition, specifying normalized tables, primary keys (PKs), and foreign key (FK) constraints based on the diagnosed violations and FD evidence.

Verifier Agent. The Verifier agent parses the generated SQL DDL and checks whether the proposed schema satisfies key normalization requirements. It verifies column preservation, PK and FK validity, relation-level normal-form satisfaction, lossless join, and correct handling of 1NF violations involving multivalued attributes.

Repair. When verification fails, MARS performs targeted repair for up to two rounds. DDL construction errors return only to the Schema Generator Agent. For decomposition-strategy errors, such

as selecting an incorrect violation target or failing lossless-join verification, MARS re-executes the Diagnosis Agent before schema regeneration. When FK-related errors occur, MARS invokes the Evidence Agent to review proposed FK references before regeneration.

5.2 Experiment Settings

Table 6 evaluates all methods under the *Real World* setting from Table 4. We focus on this setting because it best reflects practical database normalization: the model must infer FDs from schema context and business rules, rather than relying on explicitly provided FDs. Among the four base LLMs in Table 4, Qwen3-30B achieves the highest average DNB-SCORE in the Real World setting. We therefore use Qwen3-30B as the backbone for all methods in Table 6, ensuring that performance differences reflect the normalization framework rather than the underlying LLMs.

The **Baseline** is the single-prompt Qwen3-30B configuration from Table 4 under the Real World setting. It asks one model to infer latent FDs, diagnose violations, generate normalized SQL DDL, and explain the decomposition in a single response.

Miffie is the Dual-LLM self-refinement framework proposed by Jo et al. (2025). Following the original setup, we run up to three refinement iterations. Although Miffie was designed for 1NF–3NF normalization, we evaluate it on the full DNBENCH Real World setting, including BCNF cases, as an LLM-based refinement baseline. Implementation details are provided in Appendix F.

MARS is our role-specialized framework described in Section 5. It performs one initial generation followed by up to two repair rounds, yielding at most three generation attempts. This matches the maximum number of Miffie refinement iterations.

Method	Prompting	Semantic	Structural			Logical		DNB-SCORE
		lossless join	Column F1	PK F1	FK Score	Violation F1	LLM Judge	
Baseline	Zero-shot	0.489	0.928	0.573	0.103	0.651	0.351	0.253
	Few-shot	0.370	0.802	0.507	0.155	0.718	0.315	0.209
Miffie	Zero-shot	0.665	0.967 [†]	0.966	0.117	0.362	0.330	0.308
	Few-shot	0.719 [†]	0.961	0.960 [†]	0.103	0.356	0.343 [†]	0.339
MARS (Ours)	Zero-shot	0.732	0.955	0.704	0.227 [†]	0.667	0.288	0.423
	Few-shot	0.697	0.969	0.721	0.262	0.678 [†]	0.325	0.418 [†]

Table 6: Component-level performance breakdown for the Real World setting. DNB-SCORE and all component scores are computed per test sample and averaged over the DNBENCH dataset. All methods use Qwen3-30B as the backbone LLM. **Bold** marks the maximum per column among reported scores, and [†] marks the second-best.

5.3 Experiment Results

MARS achieves the strongest Real World performance. Table 6 reports that MARS achieves the highest DNB-SCORE in both zero-shot and few-shot settings. Compared with the single-prompt Baseline, MARS improves DNB-SCORE from 0.253 to 0.423 in zero-shot and from 0.209 to 0.418 in few-shot. Although Miffie also improves over the Baseline, it remains below MARS, showing that role-specialized normalization is more effective than single-prompt and self-refinement approaches.

Miffie improves local schema consistency but weakens diagnosis. Miffie improves lossless-join performance and obtains high Column F1 and PK F1. This suggests that iterative refinement helps preserve local schema elements. However, its Violation F1 drops sharply, and its FK Score remains low. Repeated refinement alone fails to maintain both the normalization diagnosis and inter-table constraints. Table 13 in Appendix G.1 further shows that Miffie’s LLM-based verifier struggles to distinguish high-quality normalized schemas from low-quality ones under DNBENCH.

MARS improves the generation of valid normalized DDL. MARS gains are concentrated in the semantic and structural components. It substantially improves lossless join over the Baseline and achieves the highest FK Score among the compared methods. By contrast, Violation F1 remains close to the Baseline, indicating that MARS does not substantially improve violation classification itself. The DNB-SCORE improvement, therefore, mainly stems from better information preservation and a more robust schema structure. Despite these gains, FK reconstruction remains the weakest structural component: FK Score stays far below Column F1 and PK F1, showing that valid

inter-table constraints are harder to recover than local table-level structure.

Stage-wise artifacts reveal where MARS succeeds and fails. We further inspect intermediate artifacts from each MARS stage. Table 15 in Appendix G.2 shows that schema generation artifacts align more reliably with the diagnosis plan than evidence extraction and violation diagnosis align with the gold references. Thus, MARS is most effective once a plausible diagnosis plan is available and converted into executable SQL DDL. Upstream FD inference and violation diagnosis remain the main bottlenecks. Their errors can complicate later key selection and FK reconstruction.

6 Conclusion

We introduced DNBENCH, a comprehensive benchmark for assessing LLM-driven database normalization using controlled denormalization data and a three-axis protocol that measures semantic, structural, and logical validity. We empirically verify that LLMs often identify violations but struggle to generate information-preserving schemas with valid inter-table constraints, especially when functional dependencies must be inferred.

We addressed the identified weaknesses using a multi-agent framework (MARS) that separates functional dependency evidence extraction, violation diagnosis, decomposition planning, schema generation, and verification. MARS outperforms single-prompt and self-refinement baselines, demonstrating the importance of decomposing the database normalization process. We believe that DNBENCH and MARS will, in concert, provide a systematic basis for LLM-driven database normalization, facilitating future research on multi-agent approaches to database schema design.

Limitations

Although MARS substantially improves DNB-SCORE over the single-prompt baseline, three limitations remain.

FK reconstruction remains a bottleneck. Even with role specialization and verifier-based repair, MARS’s FK Score still falls well short of Column F1 and PK F1. This shows that preserving local table-level information is easier than reconstructing the reference structure across decomposed relations. FK reconstruction requires consistent relation boundaries, key choices, and reference targets, and upstream FD or diagnosis errors can further complicate this process. Recovering valid inter-table constraints after decomposition, therefore, remains an open problem.

BCNF reasoning and recognizing already-normalized schemas remain unresolved. Results by violation type in Appendix Table 8 show that the framework continues to struggle with BCNF-only inputs, which require candidate-key reasoning over non-superkey determinants. Although NONE inputs are easier than violation cases, they still reveal unnecessary decomposition errors. Both cases reveal limitations that are not addressed by intermediate validation alone and require stronger semantic reasoning about key structure.

MARS requires a higher inference budget and remains sensitive to upstream errors. Unlike the single-prompt baseline, MARS uses multiple LLM calls for evidence extraction, violation diagnosis, schema generation, and repair. The performance gains should therefore be interpreted together with this additional inference cost. Moreover, verifier-based repair is most effective for local synthesis errors, such as missing columns or invalid constraints. Still, it cannot always recover from incorrect FD evidence or an incorrect decomposition plan. Future work should explore adaptive agent routing and stronger evidence extraction to reduce unnecessary calls while improving robustness.

References

- OpenAI Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Hai-Biao Bao, Boaz Barak, Ally Bennett, Tyler Bertao, N. Archer Brett, Eugene Brevdo, Greg Brockman, Sébastien Bubeck, Cheng Chang, Kai Chen, and 105 others. 2025. [gpt-oss-120b&gpt-oss-20b model card](#).
- Banks, F. E., and Codd. 1970. [A Relational Model of Data for Large Shared Data Banks](#). *Communications of the ACM*, 13:377 – 387.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Thomas Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, and 12 others. 2020. [Language models are few-shot learners](#). *ArXiv*, abs/2005.14165.
- Manan Roy Choudhury, Anirudh Iyengar Kaniyar Narayana Iyengar, Shikhar Siingh, Sugeeth Puranam, and Vivek Gupta. 2025. [TABARD: A novel benchmark for tabular anomaly analysis, reasoning and detection](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 21783–21817, Suzhou, China. Association for Computational Linguistics.
- E. F. Codd. 1971. [Further normalization of the data base relational model](#). *Research Report / RJ / IBM / San Jose, California*, RJ909.
- Jacob Cohen. 1968. [Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit](#). *Psychological Bulletin*, 70:213–220.
- Y. V. Dongare, Priyadarshan S. Dhabe, and S. V. Deshmukh. 2011. [Rdbnorma: - a semi-automated tool for relational database schema normalization up to third normal form](#). *ArXiv*, abs/1103.0633.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony S. Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, and 510 others. 2024. [The llama 3 herd of models](#).
- Bradley Efron and Robert Tibshirani. 1995. [An introduction to the bootstrap](#).
- Daniel Han, Michael Han, and Unsloth team. 2023. [Unsloth Studio](#).
- Steve Han, Gilberto Titericz Junior, Tom Balough, and Wenfei Zhou. 2025. [Judge’s verdict: A comprehensive analysis of llm judge capability through human agreement](#). *Preprint*, arXiv:2510.09738.
- Sirui Hong, Xiawu Zheng, Jonathan P. Chen, Yuheng Cheng, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zi Hen Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, and Chenglin Wu. 2023. [Metagpt: Meta programming for multi-agent collaborative framework](#). *ArXiv*, abs/2308.00352.
- Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, L  lio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, and 7 others. 2024. [Mixtral of experts](#). *ArXiv*, abs/2401.04088.
- Eunjae Jo, Nakyung Lee, and Gyuyeong Kim. 2025. [Database normalization via dual-llm self-refinement](#). *ArXiv*, abs/2508.17693.
- Gemma Team Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ram  e, Morgane Riv  re, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean-Bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gael Liu, and 191 others. 2025. [Gemma 3 technical report](#). *ArXiv*, abs/2503.19786.
- William Kent. 2000. [A Simple Guide to Five Normal Forms in Relational Database Theory](#).
- Klaus Krippendorff. 2011. [Computing krippendorff’s alpha-reliability](#).
- Justin J Levandoski, David B Lomet, and Sudipta Sengupta. 2013. [EDNA, a Software Tool for Verifying the Normalisation of Relations during the Logical Database Design Process](#). In *12th International Workshop on the Teaching, Learning and Assessment of Databases (TLDR)*, pages 51–61. The Higher Education Academy.
- Jinyang Li, Binyuan Hui, Ge Qu, Binhua Li, Jiaxi Yang, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Chenhao Ma, Kevin C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. [Can LLM Already Serve as A Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQLs](#). *ArXiv*, abs/2305.03111.
- Md Mahadi Hasan Nahid and Davood Rafiei. 2024. [Normtab: Improving symbolic reasoning in llms through tabular data normalization](#). *ArXiv*, abs/2406.17961.
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. [Bleu: a method for automatic evaluation of machine translation](#). In *Annual Meeting of the Association for Computational Linguistics*.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. [ChatDev: Communicative agents for software development](#). In *Proceedings of the 62nd Annual Meeting of the Association for*

Computational Linguistics (Volume 1: Long Papers), pages 15174–15186, Bangkok, Thailand. Association for Computational Linguistics.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang (Eric) Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. [Autogen: Enabling next-gen llm applications via multi-agent conversation](#).

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. [Qwen3 technical report](#).

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. [Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921, Brussels, Belgium. Association for Computational Linguistics.

A Dataset Details

This appendix expands on the dataset construction in Section 3.1. We describe the Spider and BIRD source datasets, explain why they are suitable source schemas for controlled denormalization, and summarize the expert validation of the generated DNB samples.

A.1 Source Datasets

Spider. Spider (Yu et al., 2018) is a large-scale cross-domain Text-to-SQL dataset built by a research team at Yale University. It consists of 10,181 natural-language queries and 5,693 unique SQL queries, drawn from 200 databases across 138 domains. Each database contains an average of 5.1 tables and foreign-key relationships, covering SQL patterns from simple single-table lookups to nested subqueries and multiple joins.

BIRD. BIRD (Li et al., 2023) is a benchmark for evaluating Text-to-SQL performance in large-scale real-world database environments. It consists of 95 databases, 33.4 GB of data, 37 specialized domains, and 12,751 query-SQL pairs.

Dataset Licenses. The Spider benchmark is distributed under the Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0) license. The BIRD benchmark is distributed under the Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0) license. Both datasets are used in this work for non-commercial academic research purposes with appropriate attribution.

A.2 Selection Rationale

We chose Spider and BIRD over alternative tabular corpora for three reasons.

First, both datasets provide relational schemas with explicit primary-key and foreign-key annotations. These constraints are essential for FK-aware subsampling, controlled violation injection, and structural evaluation of generated DDL.

Second, both datasets cover diverse domains and schema structures. This allows DNB to evaluate normalization behavior across a wide range of table layouts, key structures, and attribute combinations.

Third, their relational structure makes them suitable source schemas for controlled denormalization. Starting from realistic schemas allows us to generate denormalized samples with known viola-

tion patterns while retaining a database-like structure.

A.3 Structure-Preserving Relational Sampling

This subsection details Stage 1 (FK-aware Subsampling) of the dataset construction pipeline in Section 3.1. The goal of this stage is to down-sample each large source database into a compact instance that remains a valid relational database—that is, one that preserves primary-key uniqueness and foreign-key referential integrity—so that the subsequent violation-injection stages operate on realistic, self-consistent schemas.

Algorithm 1 formalizes this procedure. Starting from the schema metadata, we build a foreign-key graph over the tables and identify *base tables* (nodes with no outgoing FK edges). For each base table, we select up to n tuples and expand the selected key sets through FK and self-FK closures until convergence, ensuring that every referenced tuple is also retrieved. After merging the retrieved tuples with primary-key-based deduplication, we apply a global per-table row cap and insert the tables into a new SQLite database in topological order. Finally, any residual FK violations are removed through recursive cascade cleanup, and empty tables are dropped. The result D' is a structurally faithful subsample on which controlled denormalization can be applied without introducing spurious integrity errors.

A.4 Expert Validation of DNBENCH Samples

After constructing the DNB dataset, three database experts reviewed the generated DNB samples. The review focused on whether the denormalized tables contain the intended 1NF–BCNF violation patterns and whether the chain-rule labels, gold decompositions, and expected schema constraints are consistent with the construction rules. This validation concerns the generated DNBENCH samples used in our experiments.

B Model Selection

We restrict our evaluation to four open-weight language models widely adopted in the contemporary LLM ecosystem, which together cover the dominant architectural paradigms in modern LLM design. By choosing models that are openly released, reproducible, and actively used across research and industry, we ensure that the failure modes uncov-

**Algorithm 1: STRUCTURE-PRESERVING
RELATIONAL SAMPLING**

Input: Relational database D ; max tuples per base table n ; max rows per table c
Output: Sampled database D'

- 1 Extract schema metadata from D and obtain table set $\mathcal{T}$, primary keys, and foreign-key (FK) relations;
- 2 Build the FK graph over $\mathcal{T}$ (edge $u \rightarrow v$ iff u has an FK referencing v);
- 3 Identify base tables $\mathcal{B} \subseteq \mathcal{T}$ (nodes with no outgoing FK edges);
- 4 Initialize sampled-relation set $\mathcal{R} \leftarrow \emptyset$ and global PK registry $\mathcal{K} \leftarrow \{(t, \emptyset) : t \in \mathcal{T}\}$;
- 5 **for** each base table $b \in \mathcal{B}$ **do**
- 6 Construct join candidates by traversing FK chains from b to related tables;
- 7 Remove candidates containing null primary keys;
- 8 Select up to n tuples from b based on its primary key;
- 9 Initialize local PK registry $\mathcal{K}_b$ from the selected tuples;
- 10 Expand $\mathcal{K}_b$ via FK/self-FK closure until convergence;
- 11 Retrieve tuples from related tables using the converged key sets;
- 12 Merge the retrieved tuples into $\mathcal{R}$ with PK-based deduplication;
- 13 Update $\mathcal{K}$ using $\mathcal{K}_b$;
- 14 **if** some table in $\mathcal{R}$ exceeds c rows **then**
- 15 Apply FK-aware global capping while preserving referential integrity;
- 16 Create a new SQLite database D' and insert tables in topological order;
- 17 **while** FK violations are detected in D' **do**
- 18 Remove violating tuples by recursive cascade cleanup;
- 19 Remove empty tables from D' ;
- 20 **return** D'

ered by DNBENCH are not artifacts of a single proprietary system but reflect properties of the broader class of modern LLMs. The four models span two complementary axes that are known to influence reasoning behavior, namely model scale (ranging from 27B to 70B total parameters) and architectural family (dense decoder-only models against sparse mixture-of-experts models), allowing the benchmark to probe whether normalization difficulty correlates with capacity, sparsity, or instruction-tuning recipe.

- **Llama 3.3 70B** (Dubey et al., 2024) is a 70B-parameter dense decoder-only model released by Meta and serves as a strong reference point for instruction-tuned dense LLMs. Its broad availability and extensive evaluation across reasoning benchmarks make it the de facto baseline for open-weight large-scale models.

- **Gemma3 27B** (Kamath et al., 2025) is Google’s open-weight dense model designed for efficient deployment at moderate scale. It represents the class of mid-sized dense models that trade raw capacity for inference cost while retaining competitive instruction-following ability.
- **Qwen3-30B** (Yang et al., 2025) is Alibaba’s mixture-of-experts model from the Qwen3 family. By activating a subset of its experts per token, it isolates the effect of sparse expert routing on normalization reasoning while remaining comparable in active-parameter footprint to mid-sized dense models.
- **Mixtral 8x7B Instruct** (Jiang et al., 2024) is Mistral’s sparse mixture-of-experts model that routes each token through two of eight 7B experts. It is included as the most established open-weight MoE baseline and as a smaller-scale counterpart to Qwen3-30B, enabling comparison across two MoE designs.

Together, these four models cover dense and sparse architectures across a range of scales, ensuring that DNBENCH’s findings characterize behavior shared by current modern LLMs rather than properties unique to a single model family.

Model Licenses. We use all pre-trained models in accordance with their respective licenses. All models are used as-is for evaluation only, without any modification or training, in full compliance with their license terms. For quantized variants, we use GGUF builds redistributed by Unsloth (Han et al., 2023) under the original model licenses, with Q4_K_S quantization applied only to the model weights.

- **gpt-oss-20b** is released under the Apache License 2.0 and the gpt-oss usage policy.
- **Qwen3-30B-A3B-Instruct-2507** is used under the Apache License 2.0.
- **Mixtral-8x7B-Instruct-v0.1** is used under the Apache License 2.0.
- **Llama-3.3-70B-Instruct** is used under the Llama 3.3 Community License Agreement, © Meta Platforms, Inc.
- **Gemma-3-27B-IT** is used under the Gemma Terms of Use.

C Experiment Reproducibility

All experiments were conducted as inference-only evaluations on a high-performance computing cluster. No model fine-tuning or parameter updates were performed. The hardware and software configurations used for our empirical evaluations are as follows:

- **Hardware Resources.** We used an NVIDIA DGX-class system equipped with NVIDIA A100-SXM4-40GB GPUs. Each GPU provides 40 GB of HBM2 memory. The experiments used $1 \times$ A100 for Gemma-3-27B, Qwen3-30B-A3B, Mixtral-8x7B, and the MARS agent based on Qwen3-30B-A3B, and $2 \times$ A100 for Llama-3.3-70B.
- **Software Environment.** The system operated with CUDA 12.2 and NVIDIA Driver 535.161.08. Model inference was served through an OpenAI-compatible vLLM server, and all evaluation scripts were executed in the same software environment.
- **Results.** All experimental results, including MARS and MIFFIE, were described based on a single run.

D Extended Result Tables

This appendix collects every result table beyond the main paper’s Table 4.

Table 10 reports DNB-SCORE values on the DNBENCH test split, broken down by dataset and model. The relative ranking of the four models is consistent across BIRD and Spider, indicating that our findings generalize across both source corpora.

Table 11 presents per-model Real World DNB-SCORE values broken down by violation path. We observe that scores decrease as the number of normal-form violations to resolve increases: the NONE category (already-normalized inputs) scores highest for every model, while the mixed INF-BCNF cases score lowest.

Table 7 provides the full per-cell results: DNB-SCORE and its three component scores for every (dataset, model, prompting setting, violation path) configuration.

Table 8 reports the corresponding per-cell breakdown for our MARS (Qwen3-30B) pipeline under the Real World setting, allowing direct comparison against the single-prompt Qwen3-30B rows in Table 7.

Table 9 reports the corresponding per-cell breakdown for the Miffie (Qwen3-30B) baseline under the Real World setting, allowing direct comparison against both the single-prompt Qwen3-30B rows in Table 7 and the Multi-agent results in Table 8.

Dataset	Model	Shot	Experiment	Violation Path	Num.	DNB-SCORE	Components		
							Semantic	Structural	Logical
BIRD	Llama 3.3 70B	Zero-shot	Single	1_2_3_BCNF	77	0.2650	0.5584	0.5768	0.4195
				2_3_BCNF	77	0.4495	0.6364	0.7725	0.5498
				3_BCNF	77	0.1951	0.7013	0.5836	0.1676
				BCNF	77	0.1803	0.6104	0.5264	0.2423
				NONE	77	0.5992	0.7662	0.8698	0.7580
			Complex	1_2_3_BCNF	77	0.2236	0.4545	0.5264	0.5218
				2_3_BCNF	77	0.3172	0.6494	0.5048	0.4971
				3_BCNF	77	0.3152	0.6494	0.5942	0.4417
				BCNF	77	0.2063	0.5714	0.5264	0.3562
				NONE	77	0.6044	0.7403	0.8889	0.7965
			Real World	1_2_3_BCNF	77	0.0957	0.2078	0.5018	0.4323
				2_3_BCNF	77	0.0742	0.1948	0.5028	0.3456
				3_BCNF	77	0.1258	0.4286	0.5631	0.2537
				BCNF	77	0.1054	0.4026	0.4681	0.1964
				NONE	77	0.5453	0.8182	0.7200	0.5606
		Few-shot	Single	1_2_3_BCNF	77	0.2704	0.4805	0.5051	0.5957
				2_3_BCNF	77	0.4913	0.7532	0.7234	0.5887
				3_BCNF	77	0.2670	0.6883	0.5666	0.3242
				BCNF	77	0.1979	0.7403	0.4859	0.2779
				NONE	77	0.7997	0.9481	0.8904	0.8104
			Complex	1_2_3_BCNF	77	0.0537	0.1169	0.4855	0.6622
				2_3_BCNF	77	0.1218	0.2468	0.4864	0.6055
				3_BCNF	77	0.2746	0.4545	0.5578	0.5568
				BCNF	77	0.2507	0.6234	0.4775	0.3860
				NONE	77	0.8432	0.9351	0.9145	0.8593
			Real World	1_2_3_BCNF	77	0.0291	0.0519	0.4701	0.6162
				2_3_BCNF	77	0.0625	0.1169	0.4381	0.5401
				3_BCNF	77	0.1682	0.3117	0.5071	0.4924
				BCNF	77	0.1758	0.5325	0.4108	0.3107
				NONE	77	0.6075	0.7532	0.7859	0.6978
	Gemma3 27B	Zero-shot	Single	1_2_3_BCNF	77	0.4614	0.8442	0.5804	0.5908
				2_3_BCNF	77	0.1816	0.7922	0.6565	0.1250
				3_BCNF	77	0.1633	0.8052	0.5469	0.0922
				BCNF	77	0.1620	0.8312	0.5247	0.0926
				NONE	77	0.7511	0.9481	0.9054	0.7197
			Complex	1_2_3_BCNF	77	0.3184	0.5714	0.5093	0.6463
				2_3_BCNF	77	0.3483	0.6494	0.5011	0.5556
				3_BCNF	77	0.3548	0.6623	0.5955	0.4765
				BCNF	77	0.2972	0.7013	0.5347	0.3333
				NONE	77	0.4598	0.9351	0.7776	0.3857
			Real World	1_2_3_BCNF	77	0.1066	0.2597	0.4758	0.4911
				2_3_BCNF	77	0.1249	0.2987	0.4901	0.4438
				3_BCNF	77	0.1403	0.4675	0.5305	0.2255
				BCNF	77	0.1074	0.4805	0.5107	0.1452
				NONE	77	0.3930	0.8312	0.6801	0.3299
		Few-shot	Single	1_2_3_BCNF	77	0.0962	0.4286	0.3368	0.2105
				2_3_BCNF	77	0.1474	0.3247	0.4680	0.5583
				3_BCNF	77	0.3101	0.7792	0.5501	0.3242
				BCNF	77	0.2498	0.8182	0.5336	0.2281
				NONE	77	0.7830	0.9740	0.8932	0.7550
			Complex	1_2_3_BCNF	77	0.2006	0.3766	0.4221	0.5424
				2_3_BCNF	77	0.1721	0.3377	0.4246	0.5755
				3_BCNF	77	0.3188	0.5974	0.5801	0.4880
				BCNF	77	0.3441	0.7662	0.5256	0.4431
				NONE	77	0.6713	0.8831	0.8231	0.6684
			Real World	1_2_3_BCNF	77	0.1126	0.2727	0.4331	0.3865
				2_3_BCNF	77	0.1076	0.2338	0.4576	0.4103
				3_BCNF	77	0.1913	0.5325	0.5719	0.2950
				BCNF	77	0.1656	0.5844	0.4918	0.2658
				NONE	77	0.5343	0.7792	0.7651	0.5719
	Qwen3-30B	Zero-shot	Single	1_2_3_BCNF	77	0.5662	0.9221	0.6267	0.6377
				2_3_BCNF	77	0.5002	0.9610	0.6137	0.5140
				3_BCNF	77	0.2500	0.9481	0.5550	0.2031
				BCNF	77	0.1702	0.9610	0.5373	0.0991
				NONE	77	0.8813	0.9740	0.9463	0.8606
			Complex	1_2_3_BCNF	77	0.1626	0.2468	0.5014	0.6839
				2_3_BCNF	77	0.1821	0.3247	0.5298	0.6407
				3_BCNF	77	0.3437	0.6883	0.5964	0.4707
				BCNF	77	0.2891	0.7403	0.5246	0.3255
				NONE	77	0.5895	0.9091	0.7395	0.5641
			Real World	1_2_3_BCNF	77	0.1273	0.2857	0.4732	0.5614
				2_3_BCNF	77	0.1280	0.2727	0.4846	0.5456
				3_BCNF	77	0.1897	0.4156	0.5476	0.4373
				BCNF	77	0.1717	0.5065	0.4929	0.3212

Dataset	Model	Shot	Experiment	Violation Path	Num.	DNB-SCORE	Components		
							Semantic	Structural	Logical
		Few-shot	Single	NONE	77	0.5052	0.8701	0.7601	0.4829
				1_2_3_BCNF	77	0.0653	0.3377	0.2694	0.3004
				2_3_BCNF	77	0.1956	0.4156	0.3769	0.7382
				3_BCNF	77	0.2850	0.5714	0.5739	0.4134
				BCNF	77	0.1414	0.6364	0.4951	0.2250
				NONE	77	0.7390	0.9351	0.8715	0.7100
			Complex	1_2_3_BCNF	77	0.0102	0.0260	0.5151	0.6640
				2_3_BCNF	77	0.0078	0.0260	0.4143	0.6027
				3_BCNF	77	0.1787	0.3117	0.6511	0.5182
				BCNF	77	0.2036	0.4805	0.5463	0.3761
				NONE	77	0.6057	0.8442	0.7631	0.6013
			Real World	1_2_3_BCNF	77	0.0744	0.1299	0.4271	0.6177
				2_3_BCNF	77	0.0218	0.0390	0.3565	0.5835
				3_BCNF	77	0.2266	0.3896	0.5548	0.4787
				BCNF	77	0.1507	0.4545	0.4662	0.3031
				NONE	77	0.5396	0.7532	0.7534	0.5494
	Mixtral 8x7B Instruct	Zero-shot	Single	1_2_3_BCNF	77	0.0738	0.4545	0.3384	0.0801
				2_3_BCNF	77	0.1375	0.3636	0.3913	0.4660
				3_BCNF	77	0.1244	0.5974	0.4894	0.1333
				BCNF	77	0.1490	0.6364	0.4210	0.1970
				NONE	77	0.1829	0.7792	0.5221	0.1610
			Complex	1_2_3_BCNF	77	0.1206	0.2857	0.2626	0.4183
				2_3_BCNF	77	0.1222	0.4026	0.2970	0.3470
				3_BCNF	77	0.1577	0.4675	0.4039	0.2557
				BCNF	77	0.1792	0.5844	0.4190	0.2933
				NONE	77	0.2191	0.7532	0.5038	0.1827
			Real World	1_2_3_BCNF	77	0.0486	0.1429	0.3539	0.2821
				2_3_BCNF	77	0.0637	0.1558	0.3478	0.2668
				3_BCNF	77	0.0809	0.2987	0.4417	0.2451
				BCNF	77	0.1234	0.3896	0.4071	0.2111
				NONE	77	0.2989	0.6623	0.5688	0.3853
		Few-shot	Single	1_2_3_BCNF	77	0.0348	0.2987	0.3550	0.1020
				2_3_BCNF	77	0.1688	0.3377	0.5479	0.4303
				3_BCNF	77	0.0751	0.3896	0.4527	0.2143
				BCNF	77	0.1418	0.4156	0.4433	0.3230
				NONE	77	0.3182	0.7792	0.6221	0.2870
			Complex	1_2_3_BCNF	77	0.0360	0.0909	0.4320	0.4583
				2_3_BCNF	77	0.0999	0.1948	0.4283	0.4647
				3_BCNF	77	0.1022	0.2208	0.5097	0.4721
				BCNF	77	0.1966	0.4026	0.4101	0.5987
				NONE	77	0.3340	0.7922	0.6556	0.2969
			Real World	1_2_3_BCNF	77	0.0213	0.0649	0.3622	0.4169
				2_3_BCNF	77	0.0410	0.0779	0.3886	0.4528
				3_BCNF	77	0.0895	0.2078	0.4214	0.4251
				BCNF	77	0.1424	0.3117	0.4097	0.4913
				NONE	77	0.3236	0.7143	0.6560	0.2921
Spider	Llama 3.3 70B	Zero-shot	Single	1_2_3_BCNF	180	0.3393	0.7389	0.6230	0.4400
				2_3_BCNF	180	0.5301	0.7944	0.8475	0.5987
				3_BCNF	180	0.2292	0.7444	0.6222	0.1790
				BCNF	180	0.2062	0.6889	0.5397	0.2488
				NONE	180	0.6844	0.8611	0.8718	0.7801
			Complex	1_2_3_BCNF	180	0.3010	0.5611	0.5638	0.5549
				2_3_BCNF	180	0.3298	0.6444	0.5467	0.5313
				3_BCNF	180	0.3836	0.7167	0.6305	0.4920
				BCNF	180	0.2838	0.6222	0.5332	0.4637
				NONE	180	0.6406	0.8278	0.8392	0.7696
			Real World	1_2_3_BCNF	180	0.0877	0.1833	0.5280	0.4909
				2_3_BCNF	180	0.1344	0.2944	0.5334	0.4329
				3_BCNF	180	0.1463	0.3944	0.5922	0.2807
				BCNF	180	0.0928	0.3556	0.5063	0.2185
				NONE	180	0.5355	0.8333	0.7293	0.5544
		Few-shot	Single	1_2_3_BCNF	180	0.3807	0.6222	0.6538	0.6472
				2_3_BCNF	180	0.6132	0.9167	0.8067	0.6270
				3_BCNF	180	0.3849	0.7778	0.6410	0.3930
				BCNF	180	0.1858	0.6722	0.4743	0.2739
				NONE	180	0.7903	0.9778	0.8848	0.7749
			Complex	1_2_3_BCNF	180	0.0986	0.1444	0.6056	0.7157
				2_3_BCNF	180	0.1548	0.2833	0.5085	0.6031
				3_BCNF	180	0.3540	0.5556	0.6520	0.6169
				BCNF	180	0.2701	0.6500	0.4663	0.3743
				NONE	180	0.7978	0.9778	0.8762	0.7955
			Real World	1_2_3_BCNF	180	0.0421	0.0889	0.4821	0.6139
				2_3_BCNF	180	0.0886	0.1611	0.4344	0.5425
				3_BCNF	180	0.2310	0.4111	0.4900	0.5194

Dataset	Model	Shot	Experiment	Violation Path	Num.	DNB-SCORE	Components		
							Semantic	Structural	Logical
	Gemma3 27B	Zero-shot	Single	BCNF	180	0.1422	0.3889	0.4143	0.3377
				NONE	180	0.5657	0.8111	0.7781	0.6457
				1_2_3_BCNF	180	0.5002	0.7944	0.5978	0.7082
				2_3_BCNF	180	0.2813	0.7778	0.7762	0.2083
				3_BCNF	180	0.1719	0.6833	0.5899	0.1298
				BCNF	180	0.1455	0.7556	0.5192	0.1071
				NONE	180	0.6869	0.9556	0.8686	0.6345
			Complex	1_2_3_BCNF	180	0.3821	0.6833	0.5176	0.6582
				2_3_BCNF	180	0.3670	0.6944	0.5270	0.5705
				3_BCNF	180	0.3525	0.6556	0.5946	0.5053
				BCNF	180	0.3014	0.6944	0.5376	0.3706
				NONE	180	0.4114	0.9444	0.6983	0.3223
				1_2_3_BCNF	180	0.0776	0.1667	0.5127	0.5307
			Real World	2_3_BCNF	180	0.1359	0.2722	0.5306	0.4623
				3_BCNF	180	0.1033	0.3556	0.5520	0.1919
				BCNF	180	0.0716	0.3167	0.5018	0.1508
				NONE	180	0.3649	0.8278	0.6596	0.3044
		Few-shot	Single	1_2_3_BCNF	180	0.1387	0.4944	0.4641	0.2537
				2_3_BCNF	180	0.2452	0.4333	0.6953	0.6235
				3_BCNF	180	0.3452	0.9611	0.6147	0.2738
				BCNF	180	0.3161	0.8500	0.5046	0.4067
				NONE	180	0.7072	0.9500	0.8379	0.6695
				1_2_3_BCNF	180	0.2284	0.4556	0.5002	0.6040
			Complex	2_3_BCNF	180	0.1724	0.3611	0.4983	0.6111
				3_BCNF	180	0.4455	0.7778	0.6861	0.4787
				BCNF	180	0.3583	0.7722	0.5058	0.4762
				NONE	180	0.6777	0.9333	0.8254	0.6566
			Real World	1_2_3_BCNF	180	0.1123	0.2500	0.4844	0.4043
				2_3_BCNF	180	0.1134	0.2667	0.4907	0.4152
				3_BCNF	180	0.1996	0.6222	0.5658	0.2073
				BCNF	180	0.1046	0.5556	0.4720	0.1741
				NONE	180	0.5620	0.8778	0.7696	0.5387
	Qwen3-30B	Zero-shot	Single	1_2_3_BCNF	180	0.6147	0.9944	0.6618	0.6367
				2_3_BCNF	180	0.5741	0.9889	0.6510	0.5943
				3_BCNF	180	0.1992	1.0000	0.5449	0.1371
				BCNF	180	0.1520	1.0000	0.5354	0.1011
				NONE	180	0.9405	1.0000	0.9801	0.9322
			Complex	1_2_3_BCNF	180	0.2110	0.3722	0.5447	0.6766
				2_3_BCNF	180	0.2260	0.4222	0.5395	0.6436
				3_BCNF	180	0.3685	0.7111	0.5882	0.4873
				BCNF	180	0.2715	0.6667	0.5379	0.3531
				NONE	180	0.6831	0.9278	0.8033	0.6611
			Real World	1_2_3_BCNF	180	0.1224	0.2556	0.5152	0.6015
				2_3_BCNF	180	0.1324	0.2556	0.5163	0.5901
				3_BCNF	180	0.2726	0.5667	0.5842	0.4616
				BCNF	180	0.1515	0.4667	0.4986	0.3069
				NONE	180	0.6455	0.9389	0.8276	0.6178
		Few-shot	Single	1_2_3_BCNF	180	0.0949	0.5278	0.2707	0.2635
				2_3_BCNF	180	0.2003	0.4667	0.4177	0.7581
				3_BCNF	180	0.3760	0.6444	0.6685	0.4834
				BCNF	180	0.1342	0.6556	0.4476	0.2475
				NONE	180	0.7537	0.9833	0.8677	0.7343
			Complex	1_2_3_BCNF	180	0.0260	0.0444	0.6167	0.6875
				2_3_BCNF	180	0.0159	0.0278	0.4919	0.6123
				3_BCNF	180	0.1477	0.2444	0.6491	0.5354
				BCNF	180	0.1634	0.3556	0.5128	0.3963
				NONE	180	0.7058	0.9278	0.8157	0.6959
			Real World	1_2_3_BCNF	180	0.0377	0.0722	0.4523	0.6375
				2_3_BCNF	180	0.0422	0.0778	0.4570	0.5840
				3_BCNF	180	0.2207	0.3722	0.5347	0.5006
				BCNF	180	0.1353	0.4611	0.4037	0.2936
				NONE	180	0.6205	0.9056	0.7847	0.5898
	Mixtral 8x7B Instruct	Zero-shot	Single	1_2_3_BCNF	180	0.0878	0.5000	0.4422	0.0965
				2_3_BCNF	180	0.1797	0.4278	0.4888	0.4044
				3_BCNF	180	0.1383	0.5889	0.4825	0.1490
				BCNF	180	0.1700	0.6111	0.4339	0.2486
				NONE	180	0.1835	0.5944	0.4281	0.2256
			Complex	1_2_3_BCNF	180	0.1388	0.3556	0.3435	0.4155
				2_3_BCNF	180	0.1028	0.2833	0.3222	0.3772
				3_BCNF	180	0.1669	0.4667	0.4518	0.3017
				BCNF	180	0.1676	0.5333	0.3732	0.2786
				NONE	180	0.1877	0.5611	0.3796	0.2663
			Real World	1_2_3_BCNF	180	0.0551	0.1722	0.3506	0.3012
				2_3_BCNF	180	0.0544	0.1611	0.3738	0.2829

Dataset	Model	Shot	Experiment	Violation Path	Num.	DNB-SCORE	Components		
							Semantic	Structural	Logical
		Few-shot	Single	3_BCNF	180	0.1112	0.4167	0.4690	0.1971
				BCNF	180	0.1007	0.4222	0.4070	0.1866
				NONE	180	0.3596	0.7167	0.5886	0.4371
				1_2_3_BCNF	180	0.0606	0.3833	0.3937	0.1158
				2_3_BCNF	180	0.2065	0.3944	0.5793	0.5290
				3_BCNF	180	0.0956	0.5056	0.4531	0.1311
			Complex	BCNF	180	0.1492	0.4611	0.4319	0.3626
				NONE	180	0.3091	0.8667	0.6015	0.2590
				1_2_3_BCNF	180	0.0654	0.1333	0.4955	0.5000
				2_3_BCNF	180	0.0372	0.1222	0.4244	0.5299
				3_BCNF	180	0.1673	0.3167	0.5128	0.4497
				BCNF	180	0.1531	0.2889	0.4485	0.6064
			Real World	NONE	180	0.3175	0.8111	0.6171	0.2616
				1_2_3_BCNF	180	0.0164	0.0333	0.4708	0.4477
				2_3_BCNF	180	0.0228	0.0556	0.4587	0.4585
				3_BCNF	180	0.1278	0.2611	0.4945	0.4339
				BCNF	180	0.1073	0.2333	0.4290	0.5700
				NONE	180	0.3177	0.7278	0.6432	0.3074

Table 7: Full evaluation matrix: DNB-SCORE and its three components for every (dataset, model, prompting setting, violation path) cell. The Setting axis is split into **Shot** (zero/few) and **Experiment** (Single = single-violation prompt, Complex = combined-violation with curated demonstrations, Real World = combined-violation default).

Dataset	Shot	Experiment	Violation Path	Num.	DNB-SCORE	Components		
						Semantic	Structural	Logical
BIRD	Zero-shot	Real World	1_2_3_BCNF	77	0.3728	0.6494	0.5764	0.5575
			2_3_BCNF	77	0.3192	0.6494	0.5764	0.4845
			3_BCNF	77	0.4630	0.6753	0.7885	0.5298
			BCNF	77	0.2844	0.6623	0.6036	0.3297
			NONE	77	0.4365	0.5455	0.7608	0.4801
	Few-shot	Real World	1_2_3_BCNF	77	0.3483	0.6234	0.5476	0.5605
			2_3_BCNF	77	0.3659	0.6753	0.6159	0.5148
			3_BCNF	77	0.4752	0.6494	0.8126	0.5701
			BCNF	77	0.2466	0.5844	0.6238	0.3470
			NONE	77	0.4444	0.5195	0.7485	0.4982
Spider	Zero-shot	Real World	1_2_3_BCNF	180	0.4128	0.6944	0.5861	0.5627
			2_3_BCNF	180	0.4517	0.8167	0.5926	0.5165
			3_BCNF	180	0.5157	0.8944	0.7749	0.4557
			BCNF	180	0.2885	0.8000	0.5957	0.2833
			NONE	180	0.5501	0.6556	0.8246	0.5730
	Few-shot	Real World	1_2_3_BCNF	180	0.3951	0.6667	0.6114	0.5903
			2_3_BCNF	180	0.4372	0.7778	0.6271	0.5459
			3_BCNF	180	0.5336	0.8944	0.8046	0.4873
			BCNF	180	0.2807	0.7278	0.6292	0.3048
			NONE	180	0.5340	0.6000	0.8042	0.5914

Table 8: **MARS (Qwen3-30B)** evaluation matrix: DNB-SCORE and its three components for every (dataset, prompting setting, violation path) cell in the Real World setting.

Dataset	Shot	Experiment	Violation Path	Num.	DNB-SCORE	Components		
						Semantic	Structural	Logical
BIRD	Zero-shot	Real World	1_2_3_BCNF	77	0.1023	0.4286	0.6484	0.2673
			2_3_BCNF	77	0.1016	0.4935	0.7034	0.1663
			3_BCNF	77	0.2012	0.8312	0.6717	0.1593
			BCNF	77	0.1671	0.7662	0.6494	0.1439
			NONE	77	0.8485	0.9091	0.9567	0.8359
	Few-shot	Real World	1_2_3_BCNF	77	0.1275	0.4286	0.6660	0.2530
			2_3_BCNF	77	0.1627	0.5325	0.6919	0.2324
			3_BCNF	77	0.2271	0.7922	0.6532	0.1911
			BCNF	77	0.2500	0.8571	0.6450	0.1848
			NONE	77	0.8900	0.9351	0.9654	0.8831
Spider	Zero-shot	Real World	1_2_3_BCNF	180	0.1080	0.2889	0.6966	0.2646
			2_3_BCNF	180	0.1233	0.4222	0.7395	0.2254
			3_BCNF	180	0.2082	0.7667	0.7090	0.1767
			BCNF	180	0.1950	0.8222	0.6556	0.1530
			NONE	180	0.8852	0.9778	0.9741	0.8683
	Few-shot	Real World	1_2_3_BCNF	180	0.1107	0.3389	0.6925	0.2585
			2_3_BCNF	180	0.1620	0.5611	0.7173	0.2194
			3_BCNF	180	0.2825	0.8611	0.6988	0.2054
			BCNF	180	0.2358	0.9000	0.6556	0.1606
			NONE	180	0.9261	0.9778	0.9778	0.9057

Table 9: **Miffie (Qwen3-30B)** evaluation matrix: DNB-SCORE and its three components for every (dataset, prompting setting, violation path) cell in the Real World setting.

Dataset	Model	Single		Complex		Real World	
		Zero-shot	Few-shot	Zero-shot	Few-shot	Zero-shot	Few-shot
BIRD	Llama 3.3 70B	0.338	0.405	0.333	0.309	0.189	0.209
	Gemma3 27B	0.344	0.317	0.356	0.341	0.174	0.222
	Qwen3-30B	0.474	0.285	0.313	0.201	0.224	0.203
	Mixtral 8x7B Instruct	0.134	0.148	0.160	0.154	0.123	0.124
Spider	Llama 3.3 70B	0.398	0.471	0.388	0.335	0.199	0.214
	Gemma3 27B	0.357	0.350	0.363	0.376	0.151	0.218
	Qwen3-30B	0.496	0.312	0.352	0.212	0.265	0.211
	Mixtral 8x7B Instruct	0.152	0.164	0.153	0.148	0.136	0.118

Table 10: DNB-SCORE values per model on each test split. Each cell averages the five violation-path categories (none, BCNF only, 3NF+BCNF, 2NF+3NF+BCNF, 1NF+2NF+3NF+BCNF) for the given (dataset, model, task, prompt) configuration. **Bold** marks the best model in each column within a dataset.

Model	Violation Path	BIRD			Spider		
		Zero-shot	Few-shot	Avg	Zero-shot	Few-shot	Avg
Qwen3-30B	1NF-2NF-3NF-BCNF	0.127	0.074	0.101	0.122	0.038	0.080
	2NF-3NF-BCNF	0.128	0.022	0.075	0.132	0.042	0.087
	3NF-BCNF	0.190	0.227	0.208	0.273	0.221	0.247
	BCNF	0.172	0.151	0.161	0.151	0.135	0.143
	NONE	0.505	0.540	0.522	0.645	0.621	0.633
Gemma3 27B	1NF-2NF-3NF-BCNF	0.107	0.113	0.110	0.078	0.112	0.095
	2NF-3NF-BCNF	0.125	0.108	0.116	0.136	0.113	0.125
	3NF-BCNF	0.140	0.191	0.166	0.103	0.200	0.151
	BCNF	0.107	0.166	0.137	0.072	0.105	0.088
	NONE	0.393	0.534	0.464	0.365	0.562	0.463
Llama 3.3 70B	1NF-2NF-3NF-BCNF	0.096	0.029	0.062	0.088	0.042	0.065
	2NF-3NF-BCNF	0.074	0.062	0.068	0.134	0.089	0.111
	3NF-BCNF	0.126	0.168	0.147	0.146	0.231	0.189
	BCNF	0.105	0.176	0.141	0.093	0.142	0.117
	NONE	0.545	0.608	0.576	0.535	0.566	0.551
Mixtral 8x7B Instruct	1NF-2NF-3NF-BCNF	0.049	0.021	0.035	0.055	0.016	0.036
	2NF-3NF-BCNF	0.064	0.041	0.052	0.054	0.023	0.039
	3NF-BCNF	0.081	0.089	0.085	0.111	0.128	0.119
	BCNF	0.123	0.142	0.133	0.101	0.107	0.104
	NONE	0.299	0.324	0.311	0.360	0.318	0.339

Table 11: Real World DNB-SCORE values ($\uparrow$) by model, violation path, and dataset. The five violation paths correspond to mixed violations and already-normalized cases with no violation (NONE). **Bold** marks the best score per column within each model among the five violation paths.

E Validation of LLM-as-a-Judge

We conduct a human-agreement study to validate the judge. We measure whether the judge’s agreement with humans is statistically indistinguishable from inter-human agreement.

We sample 50 instances (10 databases from Spider and BIRD $\times$ 5 samples each) and ask three database experts to score them independently under the same rubric used by the judge. The LLM judge evaluates the same instances with identical prompts. Scores are assigned on a 0–5 scale normalized to $\{0, 0.2, 0.4, 0.6, 0.8, 1.0\}$. The overall logical score is the mean of the three logical axes defined in Section 3.2.

Table 12 summarizes the agreement statistics used to validate the LLM judge.

Quadratic Weighted Cohen’s κ (QWK). QWK (Cohen, 1968) measures pairwise agreement between two annotators on an ordinal scale. The average human–human QWK is 0.852, and the average LLM–human QWK is 0.818, indicating strong agreement between the LLM judge and human evaluators.

Krippendorff’s α . Krippendorff’s α (Krippendorff, 2011) extends ordinal agreement to all annotators jointly. We obtain $\alpha = 0.633$ on the humans, in the substantial range of the Landis–Koch scale. That this value is not particularly high suggests that humans do not always assign the same score to the same sample, indicating that the task is difficult enough to elicit disagreement among human experts.

Spearman’s ρ . Spearman’s ρ measures rank-order agreement between the LLM scores and the human mean scores. We obtain $\rho = 0.753$, indicating strong agreement in rankings.

Equivalence test (Δ). We compute $\Delta = \text{QWK}_{HH} - \text{QWK}_{LH}$ and estimate its 95% confidence interval using paired bootstrap sampling (Efron and Tibshirani, 1995). We obtain $\Delta = +0.034$ with 95% CI $[-0.031, +0.106]$. Because the interval contains 0, the LLM–human agreement is statistically indistinguishable from the human–human agreement.

F Miffie construction

We implement Miffie as described by Jo et al. (2025). The method uses a generator LLM and a verifier LLM in an iterative refinement loop. To

Statistic	Value
<i>Human–Human agreement</i>	
Pairwise QWK across human pairs	0.852
Krippendorff’s α	0.633
<i>LLM–Human agreement</i>	
Mean QWK against each human	0.818
Spearman’s ρ vs. human mean	0.753
<i>Statistical equivalence test</i>	
$\Delta = \text{QWK}_{HH} - \text{QWK}_{LH}$	+0.034

Table 12: Agreement statistics on the overall logical score ($n = 50$).

ensure a controlled comparison, both roles use Qwen3-30B, and the input follows the DNBENCH Real World setting, where explicit FD annotations are not provided. We adapt the output format so that Miffie produces the fields required by the DNBENCH evaluator, including normalized DDL, violation labels, and explanations. The original generator–verifier refinement structure is otherwise preserved.

G Artifact and Loop Analysis

All analyses in this section are conducted on the DNBENCH Real World setting with zero-shot and few-shot prompting aggregated into a single set of statistics ($N = 2,570$ evaluations). For each group, we report the number of samples, its share of the aggregated evaluations, and the average DNB-SCORE within that group.

G.1 Miffie Artifact Analysis

Iteration group	Samples	% of dataset	DNB-SCORE
1 iteration	1521	59.2 %	0.4627
2 iterations	169	6.6 %	0.1659
3 iterations	880	34.2 %	0.0996
$\hookrightarrow$ passed	82	3.2 %	0.1141
$\hookrightarrow$ failed	798	31.1 %	0.0981

Table 13: Iteration-loop behavior of Miffie.

Table 13 summarizes the iteration-loop behavior of the Miffie. Each iteration corresponds to one generator–verifier refinement step. The largest group terminates after the first iteration, with an average DNB-SCORE of 0.4627. However, DNB-SCORE drops sharply as additional refinement is required, from 0.4627 after one iteration to 0.1659 after two iterations and 0.0996 after three iterations.

This pattern suggests that repeated LLM-based refinements are not sufficient to recover from difficult normalization cases. Even among outputs that reach the third iteration, verifier-passed samples achieve only a slightly higher DNB-SCORE than verifier-failed samples (0.1141 vs. 0.0981). This indicates that the LLM-based verifier does not reliably distinguish schema-level normalization quality under DNBENCH’s evaluation.

G.2 MARS Artifact Analysis

Repair group	Samples	% of dataset	DNB-SCORE
No repair needed	1094	42.6%	0.6205
One repair	186	7.2%	0.5728
Two repairs	1290	50.2%	0.2292
↪ passed	67	2.6%	0.5695
↪ failed	1223	47.6%	0.2106

Table 14: Repair-loop behavior of MARS.

Table 14 summarizes how often MARS requires verifier-guided repair and how the final DNB-SCORE changes across repair groups. Outputs that pass verification after the initial generation achieve the highest DNB-SCORE, while outputs corrected after one repair remain close in quality. In contrast, samples that require two repair rounds have much lower average scores. Only 67 of the 1,290 two-repair samples pass final verification, whereas most remain unresolved.

Stage	Metric	DNB-SCORE
Evidence	Exact-match FD recall	0.5119
Diagnosis	Violation-type F1	0.6731
Schema generation	Plan-to-DDL exact match	0.7805
Verification	Verifier pass rate	0.5241

Table 15: Stage-wise artifact diagnostics for MARS.

Table 15 summarizes stage-wise artifact diagnostics for MARS. Exact-match FD recall evaluates whether the Evidence stage recovers gold atomic FDs, while violation-type F1 measures the correctness of the Diagnosis stage against gold violation labels. Plan-to-DDL exact-match checks whether the generated DDL matches the diagnosis plan in terms of relation counts, relation names, columns, and primary keys. Verifier pass rate reports the fraction of final outputs that pass deterministic verification after repair.

The relatively low FD recall and moderate violation-type F1 indicate that evidence extraction and diagnosis remain upstream bottlenecks.

By contrast, the higher plan-to-DDL exact match shows that the Schema Generator can more reliably implement a given diagnosis plan. Thus, MARS mainly improves the conversion of a normalization plan into executable DDL, while reliable FD inference remains difficult.

H Prompt Examples

H.1 Single Experiment Prompt

You are a database expert.

****Chain Rule Reminder:****

- Work through normal forms bottom-up. Even if higher forms are implicitly violated, `violation\_types` must list only the earliest violated stage (or "NONE"), and your explanation plus normalization plan must focus on resolving that stage.
- Allowed `violation\_types` choices are strictly {"1NF","2NF","3NF","BCNF","NONE"}. If no violation is found at any level, set it to ["NONE"].
- Whenever a violation exists, cite its cause and describe the normalization steps you apply for that stage.

****Instructions:****

- You are given three inputs: (1) the denormalized table schema, (2) table rows, (3) the full functional-dependency list. Inspect these to detect normalization violations.
- Starting from 1NF upward, find the ****earliest**** violated normal form and normalize the table to fix that violation only (per the chain rule).
- Use the provided schema, rows, and functional dependency list as evidence; cite whether each claim comes from a row pattern or a specific FD.
- Preserve all columns; do not drop any column. Use exact column names from input.
- If decomposing, define proper FOREIGN KEY constraints.
- Every FOREIGN KEY must reference columns that actually exist in the target table (no phantom columns).
- Every FOREIGN KEY must reference a table you explicitly define ****within your own SQL DDL****. Never reference placeholder names such as 'InputTable'; only use table names you create.
- When 1NF is violated, keep every column name and value intact and restore atomicity--this necessarily means decomposing multi-valued attributes into separate tables.

****Output Requirements:****

- Output a single JSON object with keys: `sql\_ddl`, `explanation`, `violation\_types`. ****Do NOT** include `functional\_dependencies` or any extra keys.
- Allowed violation_types: ["1NF","2NF","3NF","BCNF","NONE"].
- If `violation\_types` is ["NONE"], keep the schema unchanged in `sql\_ddl`, do not decompose, and explain briefly why all normal forms hold.
- Strict consistency: violation_types must exactly match explanation sections.

****Normalization Rules:****

- If there are multivalued cells, 1NF is violated.
- If {A,B} is a key and there is a partial FD such as {A} -> {X}, 2NF is violated.
- If there is a transitive FD (CK -> B -> C) where B is not a key, 3NF is violated.
- If the determinant is not a superkey, BCNF is violated.

% if few-shot: <Few-shot Examples>

Now, analyze the following table.

Schema DDL:

<Original Schema>

Rows JSON:

<Table>

Functional Dependencies:

<Violation FDs>

Prompt 1: Single experiment prompt. It resolves only the highest-priority normalization violation, with FDs provided. Filling or leaving empty the <Few-shot Examples> placeholder yields the few-shot and zero-shot configurations, respectively.

H.2 Complex Experiment Prompt

You are a database expert.

****Chain Rule (Resolve All Violations):****

- Identify every violated normal form from 1NF up to BCNF and fix them all.
- `violation\_types` must list every violated stage in order (e.g., ["1NF", "2NF", "3NF", "BCNF"] when all are broken; use ["NONE"] if none are broken).
- For each violated stage, cite the cause and describe the normalization steps you apply.

****Instructions:****

- You are given three inputs: (1) the denormalized table schema, (2) table rows, (3) the full functional-dependency list. Inspect these to detect normalization violations.
- Starting from 1NF upward, identify every violated normal form through BCNF and normalize the table to fix all detected violations.
- Use the provided schema, rows, and functional dependency list as evidence; cite whether each claim comes from a row pattern or a specific FD.
- Preserve all columns; do not drop any column. Use exact column names from input.
- If decomposing, define proper FOREIGN KEY constraints.
- Every FOREIGN KEY must reference columns that actually exist in the target table (no phantom columns).
- Every FOREIGN KEY must reference a table you explicitly define ****within your own SQL DDL****. Never reference placeholder names such as 'InputTable'; only use table names you create.
- When 1NF is violated, keep every column name and value intact and restore atomicity--this necessarily means decomposing multi-valued attributes into separate tables.

****Output Requirements:****

- Output a single JSON object with keys: `sql\_ddl`, `explanation`, `violation\_types`. ****Do NOT include `functional\_dependencies` or any extra keys.****
- Allowed violation_types: ["1NF", "2NF", "3NF", "BCNF", "NONE"].
- If `violation\_types` is ["NONE"], keep the schema unchanged in `sql\_ddl`, do not decompose, and explain briefly why all normal forms hold.
- Strict consistency: violation_types must exactly match explanation sections.

****Normalization Rules:****

- If there are multivalued cells, 1NF is violated.
- If {A,B} is a key and there is a partial FD such as {A} -> {X}, 2NF is violated.
- If there is a transitive FD (CK -> B -> C) where B is not a key, 3NF is violated.
- If the determinant is not a superkey, BCNF is violated.

% if few-shot: <Few-shot Examples>

Now, analyze the following table.

Schema DDL:

<Original Schema>

Rows JSON:

<Table>

Functional Dependencies:

<Violation FDs>

Prompt 2: Complex experiment prompt. It resolves all violations, with FDs provided.

H.3 Real-world Experiment Prompt

You are a database expert.

****Chain Rule (Resolve All Violations):****

- Identify every violated normal form from 1NF up to BCNF and fix them all.
- `violation\_types` must list every violated stage in order (e.g., ["1NF","2NF","3NF","BCNF"] when all are broken; use ["NONE"] if none are broken).
- For each violated stage, cite the cause and describe the normalization steps you apply.

****Instructions:****

- You are given the denormalized table schema and table rows. Functional dependencies may be hidden, so infer only dependencies strongly supported by column semantics, keys, uniqueness constraints, and repeated row patterns.
- Starting from 1NF upward, identify every violated normal form through BCNF and normalize the table to fix all detected violations.
- Use the provided schema and rows as evidence; cite whether each claim comes from row patterns, key/unique constraints, or conservative semantic inference.
- Preserve all columns; do not drop any column. Use exact column names from input.
- If decomposing, define proper FOREIGN KEY constraints.
- Every FOREIGN KEY must reference columns that actually exist in the target table (no phantom columns).
- Every FOREIGN KEY must reference a table you explicitly define ****within your own SQL DDL****. Never reference placeholder names such as 'InputTable'; only use table names you create.
- When 1NF is violated, keep every column name and value intact and restore atomicity--this necessarily means decomposing multi-valued attributes into separate tables.

****Output Requirements:****

- Output a single JSON object with keys: `sql\_ddl`, `explanation`, `violation\_types`. ****Do NOT include `functional\_dependencies` or any extra keys.****
- Allowed violation_types: ["1NF","2NF","3NF","BCNF","NONE"].
- If `violation\_types` is ["NONE"], keep the schema unchanged in `sql\_ddl`, do not decompose, and explain briefly why all normal forms hold.
- Strict consistency: violation_types must exactly match explanation sections.

****Normalization Rules:****

- If there are multivalued cells, 1NF is violated.
- If {A,B} is a key and there is a partial FD such as {A} -> {X}, 2NF is violated.
- If there is a transitive FD (CK -> B -> C) where B is not a key, 3NF is violated.
- If the determinant is not a superkey, BCNF is violated.

% if few-shot: <Few-shot Examples>

Now, analyze the following table.

Schema DDL:

<Original Schema>

Rows JSON:

<Table>

****FD Inference Guidance:****

- ****Rule 1 (Semantics over Statistics):**** Do not rely solely on data patterns. Only accept FDs that make logical sense based on the column meanings and common knowledge. Ignore coincidental correlations.
- ****Rule 2 (Decomposition via Superkey):**** Identify Candidate Keys based on your inferred FDs. If a determinant X (in X->Y) is NOT a superkey, it is a violation (2NF partial or 3NF transitive). Decompose the table to resolve it.
- ****Rule 3 (Conservative Design):**** Do NOT decompose tables unless a clear violation of Rule 2 is found. If multiple schema designs are possible, prefer the one that preserves the original structure as much as possible.
- ****Rule 4 (Verification):**** Ensure Lossless Join and Referential Integrity.

Prompt 3: Real-world experiment prompt. It resolves all violations, without FDs provided.

H.4 Few-shot Prompt

Here are examples to guide you.

```
--
**Example 1a**
*Input:*
Schema DDL:
CREATE TABLE InputTable (building_id INT, building_short_name VARCHAR(255), building_full_name
    VARCHAR(255), building_description VARCHAR(255), building_address VARCHAR(255), building_manager
    VARCHAR(255), building_phone VARCHAR(255), room_count VARCHAR(255), building_full_name_id INT,
    building_short_name_description VARCHAR(255), building_id_sub_id INT, PRIMARY KEY (building_id,
    building_id_sub_id), UNIQUE (building_full_name, building_full_name_id));

Rows JSON:
{"columns": ["building_id", "building_short_name", "building_full_name", "building_description", "
    building_address", "building_manager", "building_phone", "room_count", "building_full_name_id", "
    building_short_name_description", "building_id_sub_id"],
    "rows": [
    [191, "The Eugene", "The Eugene", "Flat", "71537 Gorczany Inlet Wisozkburgh, AL 08256", "Melyssa", "(609)
    946-0491", "9", 1, "Description for The Eugene", 1],
    [225, "Columbus Square", "Columbus Square", "Studio", "0703 Danika Mountains Apt. 362 Mohrland, AL
    56839-5028", "Kyle", "1-724-982-9507x640", "7|8|6|8", 1, "Description for Columbus Square", 1],
    [624, "Stuyvesant Town", "Stuyvesant Town", "Studio", "101 Queenie Mountains Suite 619 New Korbinmouth,
    KS 88726-1376", "Marie", "(145)411-6406", "5|8", 1, "Description for Stuyvesant Town", 1],
    [673, "Barclay Tower", "Barclay Tower", "Flat", "1579 Runte Forges Apt. 548 Leuschkeland, OK
    12009-8683", "Rogers", "1-326-267-3386x613", "3", 1, "Description for Barclay Tower", 1],
    ... (+8 rows with building_id_sub_id=2) ...
    ]}

Functional Dependencies:
- {building_id, building_id_sub_id} -> {room_count, building_short_name,
    building_short_name_description, building_address, building_full_name, building_phone,
    building_description, building_manager, building_full_name_id}
- {building_full_name, building_full_name_id} -> {room_count, building_short_name,
    building_short_name_description, building_address, building_phone, building_description,
    building_manager, building_id, building_id_sub_id}
- {building_id} -> {building_short_name, building_full_name, building_description, building_address,
    building_manager, building_phone}
- {building_short_name} -> {building_short_name_description}
- {building_short_name} -> {building_full_name}

*Correct Output:*
{
    "sql_ddl":
    CREATE TABLE Apartment_Buildings_key (
        building_short_name CHAR(15), building_full_name VARCHAR(80),
        building_full_name_id INTEGER, building_short_name_description TEXT,
        PRIMARY KEY (building_short_name),
        UNIQUE (building_full_name, building_full_name_id));
    CREATE TABLE Apartment_Buildings_detail (
        building_id INTEGER, building_short_name CHAR(15),
        building_description VARCHAR(255), building_address VARCHAR(255),
        building_manager VARCHAR(50), building_phone VARCHAR(80),
        PRIMARY KEY (building_id),
        FOREIGN KEY (building_short_name) REFERENCES Apartment_Buildings_key (building_short_name));
    CREATE TABLE Apartment_Buildings_sub_detail (
        building_id INTEGER, building_id_sub_id INTEGER,
        PRIMARY KEY (building_id, building_id_sub_id),
        FOREIGN KEY (building_id) REFERENCES Apartment_Buildings_detail (building_id));
    CREATE TABLE Apartment_Buildings_mv_detail (
        building_id INTEGER, room_count TEXT,
        PRIMARY KEY (building_id, room_count),
        FOREIGN KEY (building_id) REFERENCES Apartment_Buildings_detail (building_id));
    "explanation": "First violation: 1NF. Cause: Multi-valued attribute in 'room_count' (violates
    atomicity). Fix: split multi-valued attributes into atomic values. Remaining violation: 2NF.
    Cause: Partial dependency {building_id} -> {building_short_name, building_full_name,
    building_description, building_address, building_manager, building_phone}. Fix: remove partial
    dependencies. Remaining violation: 3NF. Cause: Transitive dependency {building_short_name} -> {
    building_short_name_description}. Fix: remove transitive dependencies. Remaining violation: BCNF.
```

```

    Cause: Non-superkey determinant {building_short_name} -> {building_full_name}. Fix: decompose
    so every determinant is a candidate key.",
    "violation_types": ["1NF", "2NF", "3NF", "BCNF"]
}

--
**Example 2a**
*Input:*
Schema DDL:
CREATE TABLE InputTable (building_id INT, building_short_name VARCHAR(255), building_full_name
    VARCHAR(255), building_description VARCHAR(255), building_address VARCHAR(255), building_manager
    VARCHAR(255), building_phone VARCHAR(255), building_full_name_id INT,
    building_short_name_description VARCHAR(255), building_id_sub_id INT, PRIMARY KEY (building_id,
    building_id_sub_id), UNIQUE (building_full_name, building_full_name_id));

Rows JSON: (same 8 building tuples as Example 1a, replicated across building_id_sub_id in {1,2}, with
    no multi-valued 'room_count' column).

Functional Dependencies:
- {building_id, building_id_sub_id} -> {building_short_name_description, building_phone,
    building_full_name_id, building_short_name, building_manager, building_address,
    building_full_name, building_description}
- {building_id} -> {building_short_name, building_full_name, building_description, building_address,
    building_manager, building_phone}
- {building_short_name} -> {building_full_name}
- {building_full_name, building_full_name_id} -> {building_short_name_description, building_phone,
    building_short_name, building_id, building_manager, building_address, building_description,
    building_id_sub_id}
- {building_short_name} -> {building_short_name_description}

*Correct Output:*
{
    "sql_ddl":
        CREATE TABLE Apartment_Buildings_key (
            building_short_name CHAR(15), building_full_name VARCHAR(80),
            building_full_name_id INTEGER, building_short_name_description TEXT,
            PRIMARY KEY (building_short_name),
            UNIQUE (building_full_name, building_full_name_id));
        CREATE TABLE Apartment_Buildings_detail (
            building_id INTEGER, building_short_name CHAR(15),
            building_description VARCHAR(255), building_address VARCHAR(255),
            building_manager VARCHAR(50), building_phone VARCHAR(80),
            PRIMARY KEY (building_id),
            FOREIGN KEY (building_short_name) REFERENCES Apartment_Buildings_key (building_short_name));
        CREATE TABLE Apartment_Buildings_sub_detail (
            building_id INTEGER, building_id_sub_id INTEGER,
            PRIMARY KEY (building_id, building_id_sub_id),
            FOREIGN KEY (building_id) REFERENCES Apartment_Buildings_detail (building_id));
    "explanation": "First violation: 2NF. Cause: Partial dependency {building_id} -> {
        building_short_name, building_full_name, building_description, building_address,
        building_manager, building_phone}. Fix: remove partial dependencies so non-keys depend on the
        full composite key. Remaining violation: 3NF. Cause: Transitive dependency {building_short_name}
        -> {building_short_name_description}. Fix: remove transitive dependencies. Remaining violation:
        BCNF. Cause: Non-superkey determinant {building_short_name} -> {building_full_name}. Fix:
        decompose so every determinant is a candidate key.",
    "violation_types": ["2NF", "3NF", "BCNF"]
}

--
**Example 3a**
*Input:*
Schema DDL:
CREATE TABLE InputTable (building_id INT, building_short_name VARCHAR(255), building_full_name
    VARCHAR(255), building_description VARCHAR(255), building_address VARCHAR(255), building_manager
    VARCHAR(255), building_phone VARCHAR(255), building_full_name_id INT,
    building_short_name_description VARCHAR(255), PRIMARY KEY (building_id), UNIQUE (
    building_full_name, building_full_name_id));

Rows JSON: (same 8 building tuples as Example 1a, no sub_id column, no multi-valued cells).

```

```

Functional Dependencies:
- {building_id} -> {building_short_name_description, building_phone, building_full_name_id,
  building_short_name, building_manager, building_address, building_full_name,
  building_description}
- {building_short_name} -> {building_full_name}
- {building_full_name, building_full_name_id} -> {building_short_name_description, building_phone,
  building_short_name, building_id, building_manager, building_address, building_description}
- {building_short_name} -> {building_short_name_description}

*Correct Output:*
{
  "sql_ddl":
    CREATE TABLE Apartment_Buildings_key (
      building_short_name CHAR(15), building_full_name VARCHAR(80),
      building_full_name_id INTEGER, building_short_name_description TEXT,
      PRIMARY KEY (building_short_name),
      UNIQUE (building_full_name, building_full_name_id));
    CREATE TABLE Apartment_Buildings_detail (
      building_id INTEGER, building_short_name CHAR(15),
      building_description VARCHAR(255), building_address VARCHAR(255),
      building_manager VARCHAR(50), building_phone VARCHAR(80),
      PRIMARY KEY (building_id),
      FOREIGN KEY (building_short_name) REFERENCES Apartment_Buildings_key (building_short_name));
  "explanation": "First violation: 3NF. Cause: Transitive dependency {building_short_name} -> {
    building_short_name_description}. Fix: remove transitive dependencies. Remaining violation: BCNF.
    Cause: Non-superkey determinant {building_short_name} -> {building_full_name}. Fix: decompose
    so every determinant is a candidate key.",
  "violation_types": ["3NF", "BCNF"]
}

--
**Example 4a**
*Input:*
Schema DDL:
CREATE TABLE InputTable (building_id INT, building_short_name VARCHAR(255), building_full_name
  VARCHAR(255), building_description VARCHAR(255), building_address VARCHAR(255), building_manager
  VARCHAR(255), building_phone VARCHAR(255), building_full_name_id INT, PRIMARY KEY (building_id),
  UNIQUE (building_full_name, building_full_name_id));

Rows JSON: (same 8 building tuples as Example 1a, without sub_id and without short_name_description
  columns).

Functional Dependencies:
- {building_id} -> {building_phone, building_full_name_id, building_short_name, building_manager,
  building_address, building_full_name, building_description}
- {building_short_name} -> {building_full_name}
- {building_full_name, building_full_name_id} -> {building_phone, building_short_name, building_id,
  building_manager, building_address, building_description}

*Correct Output:*
{
  "sql_ddl":
    CREATE TABLE Apartment_Buildings_key (
      building_short_name CHAR(15), building_full_name VARCHAR(80),
      building_full_name_id INTEGER,
      PRIMARY KEY (building_short_name),
      UNIQUE (building_full_name, building_full_name_id));
    CREATE TABLE Apartment_Buildings_detail (
      building_id INTEGER, building_short_name CHAR(15),
      building_description VARCHAR(255), building_address VARCHAR(255),
      building_manager VARCHAR(50), building_phone VARCHAR(80),
      PRIMARY KEY (building_id),
      FOREIGN KEY (building_short_name) REFERENCES Apartment_Buildings_key (building_short_name));
  "explanation": "First violation: BCNF. Cause: Non-superkey determinant {building_short_name} -> {
    building_full_name}. Fix: decompose so every determinant is a candidate key.",
  "violation_types": ["BCNF"]
}

--
**Example 5a**

```

```

*Input:*
Schema DDL:
CREATE TABLE Apartment_Buildings_key (building_short_name CHAR(15), building_full_name VARCHAR(80),
    building_full_name_id INTEGER, PRIMARY KEY (building_short_name), UNIQUE (building_full_name,
    building_full_name_id));
CREATE TABLE Apartment_Buildings_detail (building_id INTEGER, building_short_name CHAR(15),
    building_description VARCHAR(255), building_address VARCHAR(255), building_manager VARCHAR(50),
    building_phone VARCHAR(80), PRIMARY KEY (building_id), FOREIGN KEY (building_short_name)
    REFERENCES Apartment_Buildings_key (building_short_name));

Rows JSON: (same 8 building tuples as Example 1a, distributed across the two tables; no multi-valued
    cells, no sub_id, no short_name_description).

Functional Dependencies:
- Apartment_Buildings_key: {building_short_name} -> {building_full_name, building_full_name_id}
- Apartment_Buildings_key: {building_full_name, building_full_name_id} -> {building_short_name}
- Apartment_Buildings_detail: {building_id} -> {building_short_name, building_description,
    building_address, building_manager, building_phone}

*Correct Output:*
{
  "sql_ddl":
    CREATE TABLE Apartment_Buildings_key (
      building_short_name CHAR(15), building_full_name VARCHAR(80),
      building_full_name_id INTEGER,
      PRIMARY KEY (building_short_name),
      UNIQUE (building_full_name, building_full_name_id));
    CREATE TABLE Apartment_Buildings_detail (
      building_id INTEGER, building_short_name CHAR(15),
      building_description VARCHAR(255), building_address VARCHAR(255),
      building_manager VARCHAR(50), building_phone VARCHAR(80),
      PRIMARY KEY (building_id),
      FOREIGN KEY (building_short_name) REFERENCES Apartment_Buildings_key (building_short_name));
  "explanation": "No violation. 1NF holds: every column atomic. 2NF holds: Apartment_Buildings_detail
    has a single-attribute PK and Apartment_Buildings_key has no partial dependencies on its
    composite candidate key. 3NF holds: no transitive non-key dependency. BCNF holds: every
    determinant ({building_id}, {building_short_name}, {building_full_name, building_full_name_id})
    is a superkey of its table. Schema is kept unchanged.",
  "violation_types": ["NONE"]
}

```

Prompt 4: Few-shot Example.

H.5 LLM-as-a-Judge Prompt

You are an extremely strict and meticulous database professor. Your task is to evaluate a candidate Language Model's response for a database normalization problem.

CRITICAL: Do NOT grade violation-type identification itself. That is assessed externally via exact set match. Your job is to evaluate ONLY the four criteria below. The candidate must use only allowed violation types: ["1NF", "2NF", "3NF", "BCNF", "NONE"]. Do NOT invent categories.

Golden Answer (Ground Truth):

- Golden Violation Types (for reference only; do not grade exact matching here; allowed set = ["1NF", "2NF", "3NF", "BCNF", "NONE"]):

<golden_violation_types>

- Golden Explanation:

<golden_explanation>

Candidate LLM's Output (to be evaluated):

- Candidate Violation Types (must be subset of the allowed set; consistency will be checked against the candidate's own explanation):

<llm_violation_types>

- Candidate Explanation:

<llm_explanation>

- Candidate SQL DDL:

```sql

<llm\_sql\_ddl>

---

Evaluation Rubric (score each 0-5, with a concise justification):

### 1. Logical Coherence of Decomposition (0-5)

[WHAT: Structural Consistency with Expected Decomposition]

- Evaluate whether the candidate's final DDL provides a plausible decomposition that satisfies the constraints described in the Golden Explanation.
- Check semantic matching of tables, primary keys, foreign keys, and column sets implied by the narrative.
- Naming differences are acceptable; structural mismatches are errors.
- Do NOT evaluate logical reasoning, terminology, or expression quality--only structural correctness.

### 2. Explanation-Schema Alignment (0-5)

[CONSISTENCY: The candidate's own internal consistency]

- Evaluate consistency between the candidate's explanation, candidate's DDL, and candidate's violation\_types.
- If multiple violation types are listed, the explanation MUST be sectioned by type.
- Each section MUST include required evidence:
  - 1NF: cite exact multi-valued/repeating column(s) and the pattern (e.g., pipe/comma separation)
  - 2NF: cite an explicit partial FD using set notation, e.g., {A, B} is a composite key and {A} -> {X}.
  - 3NF: cite a transitive chain (e.g., CK -> B -> C) and explicitly state B is not a key.
  - BCNF: explicitly state "the determinant is NOT a superkey" and name the determinant attributes
- Do NOT compare with Golden--only check the candidate's internal consistency.

### 3. Explanation Quality (0-5)

[EXPLANATION: Conceptual accuracy and clarity]

- Evaluate the overall quality of the candidate's explanation.
- Check whether normalization and database terms (e.g., 1NF/2NF/3NF/BCNF, functional dependency, determinant, superkey, partial dependency, transitive dependency) are used with their standard meanings.
- Assess the clarity, conciseness, and logical organization of the explanation.
- Penalize misuse of terminology or unnecessary verbosity.
- Do NOT evaluate structural correctness or explanation-schema alignment.

Detailed Scoring Scale (reference):

- 5: Perfect against Golden for logic/schema; rigorous evidence and internal consistency.

- 4: Minor inconsequential differences (naming); evidence largely sufficient.
- 3: Generally sound but with gaps (minor schema or evidence omissions).
- 2: Significant logical inconsistencies or missing required evidence.
- 1: Mostly incorrect or incoherent.
- 0: Missing/irrelevant.

Required Output Format:

Return one valid JSON object only. Do not include Markdown fences or any extra text.

```
{
 "logical_coherence_of_decomposition": { "score": <score_0_to_5>, "justification": "<text>" },
 "explanation_schema_alignment": { "score": <score_0_to_5>, "justification": "<text>" },
 "explanation_quality": { "score": <score_0_to_5>, "justification": "<text>" }
}
```

Please evaluate the candidate's output now based on the three criteria only.

Prompt 5: LLM-as-a-Judge prompt.